

Graph, BFS & DFS in Java

Adjacency Matrix and Adjacency List (Linked List)

1. Graph using Adjacency Matrix

```
class GraphMatrix {  
  
    int[][] adj;  
    int vertices;  
  
    GraphMatrix(int vertices) {  
        this.vertices = vertices;  
        adj = new int[vertices][vertices];  
    }  
  
    void addEdge(int u, int v) {  
        adj[u][v] = 1;  
        adj[v][u] = 1;    // Undirected graph  
    }  
  
    void display() {  
        for (int i = 0; i < vertices; i++) {  
            for (int j = 0; j < vertices; j++) {  
                System.out.print(adj[i][j] + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

2. Graph using Adjacency List with Linked List

```
class GraphList {  
  
    class Node {  
        int vertex;  
        Node next;  
  
        Node(int vertex) {  
            this.vertex = vertex;  
        }  
    }  
  
    Node[] adjList;  
    int vertices;  
  
    GraphList(int vertices) {  
        this.vertices = vertices;  
        adjList = new Node[vertices];  
    }  
  
    void addEdge(int u, int v) {  
  
        Node newNode = new Node(v);  
        newNode.next = adjList[u];  
        adjList[u] = newNode;  
  
        newNode = new Node(u);  
        newNode.next = adjList[v];  
        adjList[v] = newNode;  
    }  
  
    void display() {  
        for (int i = 0; i < vertices; i++) {  
  
            System.out.print(i + " -> ");  
  
            Node temp = adjList[i];  
  
            while (temp != null) {  
                System.out.print(temp.vertex + " ");  
                temp = temp.next;  
            }  
  
            System.out.println();  
        }  
    }  
}
```

} } }

3. BFS using Adjacency Matrix

```
void BFS(int start) {  
    boolean[] visited = new boolean[vertices];  
  
    int[] queue = new int[vertices];  
    int front = 0;  
    int rear = 0;  
  
    visited[start] = true;  
    queue[rear++] = start;  
  
    while (front < rear) {  
        int current = queue[front++];  
  
        System.out.print(current + " ");  
  
        for (int i = 0; i < vertices; i++) {  
            if (adj[current][i] == 1 && !visited[i]) {  
                visited[i] = true;  
                queue[rear++] = i;  
            }  
        }  
    }  
}
```

4. BFS using Adjacency List (Linked List)

```
void BFS(int start) {  
    boolean[] visited = new boolean[vertices];  
  
    int[] queue = new int[vertices];  
    int front = 0;  
    int rear = 0;  
  
    visited[start] = true;  
    queue[rear++] = start;  
  
    while (front < rear) {  
        int current = queue[front++];  
  
        System.out.print(current + " ");  
  
        Node temp = adjList[current];  
  
        while (temp != null) {  
            int neighbor = temp.vertex;  
  
            if (!visited[neighbor]) {  
                visited[neighbor] = true;  
                queue[rear++] = neighbor;  
            }  
  
            temp = temp.next;  
        }  
    }  
}
```

5. DFS Recursive using Adjacency Matrix

```
void DFS(int current, boolean[] visited) {  
    visited[current] = true;  
  
    System.out.print(current + " ");  
  
    for (int i = 0; i < vertices; i++) {  
        if (adj[current][i] == 1 && !visited[i]) {  
            DFS(i, visited);  
        }  
    }  
}
```

```
    }  
}
```

6. DFS Iterative using Adjacency Matrix

```
void DFSIterative(int start) {  
    boolean[] visited = new boolean[vertices];  
  
    int[] stack = new int[vertices];  
    int top = -1;  
  
    stack[++top] = start;  
  
    while (top >= 0) {  
        int current = stack[top--];  
  
        if (!visited[current]) {  
            visited[current] = true;  
            System.out.print(current + " ");  
  
            for (int i = vertices - 1; i >= 0; i--) {  
                if (adj[current][i] == 1 && !visited[i]) {  
                    stack[++top] = i;  
                }  
            }  
        }  
    }  
}
```

7. DFS Recursive using Adjacency List

```
void DFS(int current, boolean[] visited) {  
    visited[current] = true;  
    System.out.print(current + " ");  
    Node temp = adjList[current];  
    while (temp != null) {  
        int neighbor = temp.vertex;  
        if (!visited[neighbor]) {  
            DFS(neighbor, visited);  
        }  
        temp = temp.next;  
    }  
}
```

8. DFS Iterative using Adjacency List

```
void DFSIterative(int start) {  
    boolean[] visited = new boolean[vertices];  
    int[] stack = new int[vertices];  
    int top = -1;  
    stack[++top] = start;  
    while (top >= 0) {  
        int current = stack[top--];  
        if (!visited[current]) {  
            visited[current] = true;  
            System.out.print(current + " ");  
            Node temp = adjList[current];  
            while (temp != null) {  
                if (!visited[temp.vertex]) {  
                    stack[++top] = temp.vertex;  
                }  
                temp = temp.next;  
            }  
        }  
    }  
}
```

Quick Reference

BFS: Queue | **DFS Recursive:** Function Call Stack | **DFS Iterative:** Explicit Stack

Matrix neighbor check: `adj[current][i] == 1`

Linked-list neighbor check: traverse `adjList[current]` using `Node.next`